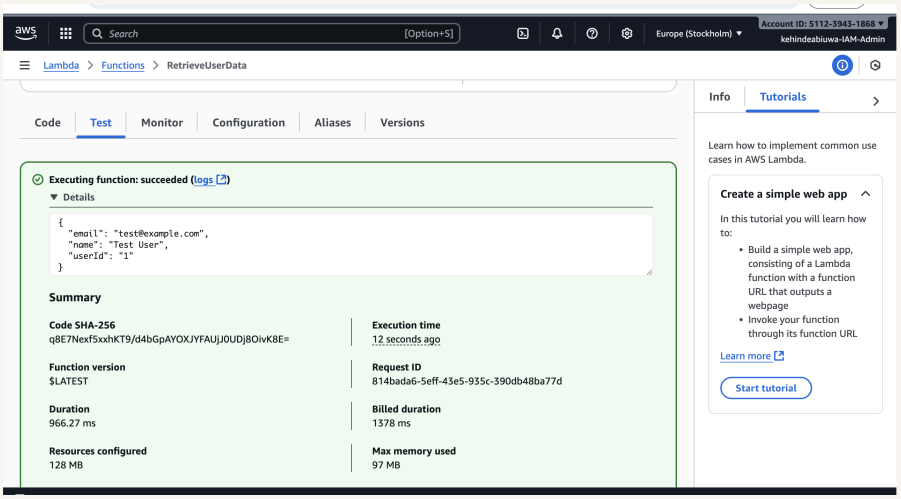
nextwork.org

Fetch Data with AWS Lambda

Kehinde Abiuwa



Kehinde Abiuwa
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate how to set up a DynamoDB table for user data, build an AWS Lambda function that fetches that data, write tests to prove it works, and lock everything down with least-privilege IAM permissions (including an inline policy on the table). I'm doing this project to learn the core serverless flow for the data tier of a three-tier architecture: modeling data in DynamoDB, reading it with Lambda, testing it, and securing it properly.

Tools and concepts

Services I used were AWS Lambda, Amazon DynamoDB, IAM, and CloudWatch — plus the AWS SDK for JavaScript (v3) in the code. Key concepts I learnt include Lambda functions, serverless on-demand scaling, execution roles & least-privilege IAM, managed vs inline policies, scoping permissions to a table ARN, DynamoDB data modeling (schemaless, partition key userId), using the DocumentClient GetItem, environment variables (TABLE_NAME), testing in the Lambda console, and reading CloudWatch logs to troubleshoot errors like AccessDenied.

Project reflection

This project took me approximately 40 minutes from creating the UserData table to deploying and testing the RetrieveUserData Lambda. The most challenging part was diagnosing the "Execution succeeded but access denied" issue and then tightening IAM by replacing the broad read-only policy with a precise inline policy that only allows dynamodb:GetItem on the UserData table. It was most rewarding to re-run the test and see the green success banner return the exact item (userId: "1")—knowing it works end-to-end and follows least-privilege best practices.

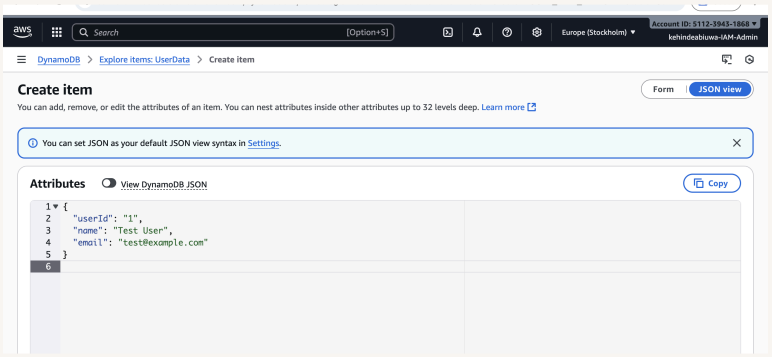
I chose to do this project today because... Something that would make learning with NextWork even better is...

Kehinde Abiuwa
NextWork Student

Project Setup

To set up my project, I created a database using Amazon DynamoDB named UserData. The partition key is userId (String), which means each record is uniquely identified and retrieved by its user ID for fast, direct lookups (no sort key needed). I left the rest of the settings at their defaults (on-demand capacity, default encryption) and created the table.

In my DynamoDB table, I added a sample user item: userId: "1", name: "Test User", and email: "test@example.com". DynamoDB is schemaless, which means you don't predefine columns—items in the same table can have different attributes, and only the key (userId) is required. You can add or change fields later without migrations.



AWS Lambda

AWS Lambda is a service that lets you run code without needing to manage any computers/servers - Lambda will manage them for you. Lambda runs your code only when you need it to (so you're not paying for any idle time). It also scales automatically, from a few requests per day to thousands per second - all you need to do is supply your code in one of the languages that Lambda supports.

I'm using Lambda in this project to run a small function that fetches user records from the UserData DynamoDB table on demand—so I only pay when it runs and it scales automatically.

Kehinde Abiuwa
NextWork Student

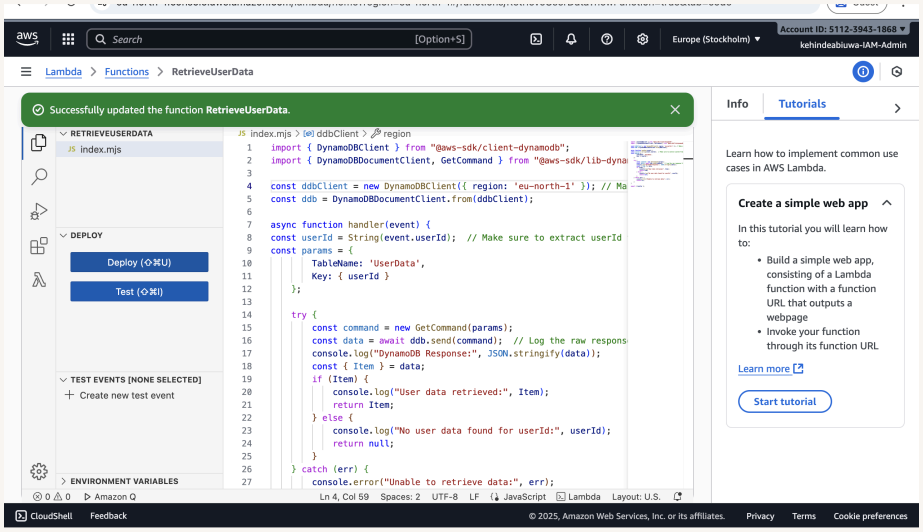
AWS Lambda Function

My Lambda function has an execution role, which is an IAM role the function assumes at runtime so it can call AWS services on my behalf. By default, the role grants CloudWatch Logs permissions (so the function can write logs), and it has a trust policy that allows lambda.amazonaws.com to assume the role.

My Lambda function will read a single user record from the UserData DynamoDB table. It: Creates a DynamoDB client (in your region) and a DocumentClient. Extracts userId from the event input. Runs a GetCommand on UserData with that key. Logs DynamoDB's raw response for troubleshooting. Returns the item if found, or null if it doesn't exist. Catches and logs any errors, returning the error object.

The code uses AWS SDK, which is Amazon's official library for calling AWS services from your code. It handles things like authentication with IAM, request signing, retries, serialization, and pagination, so you don't have to. My code uses SDK to: Create a DynamoDB client (DynamoDBClient) and a higher-level Document Client (DynamoDBDocumentClient) so I can read/write plain JavaScript objects instead of DynamoDB's low-level types. Build and send a GetCommand to the UserData table to fetch a user by userId. Automatically use the Lambda execution role's credentials and the specified region, with built-in retries and error handling.

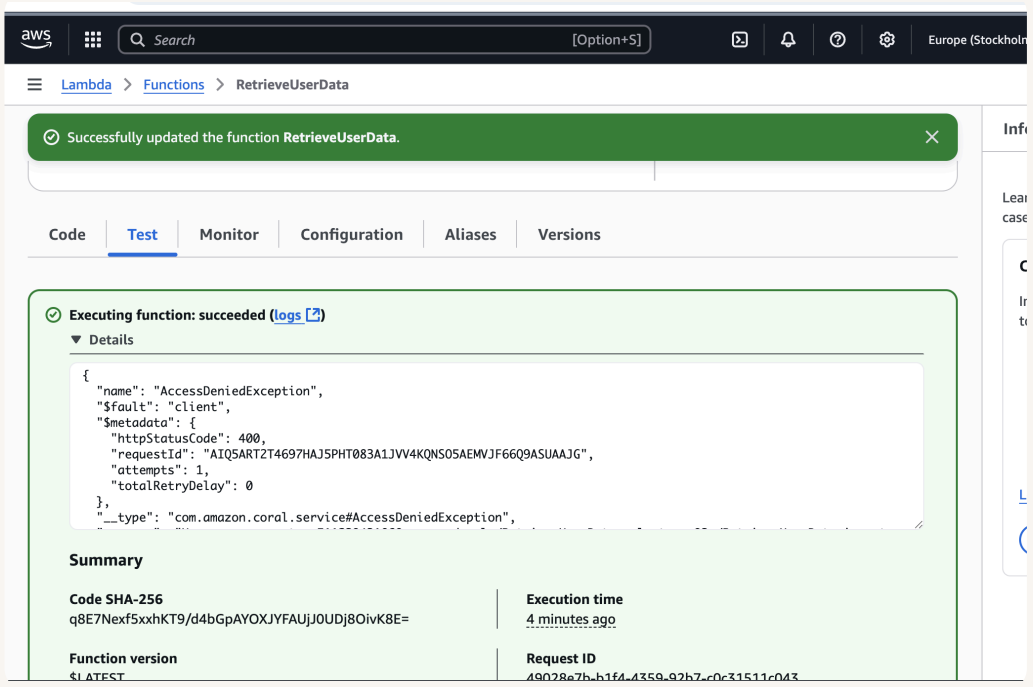
Kehinde Abiuwa
NextWork Student



Function Testing

To test whether my Lambda function works, I created a test event in the Lambda console and invoked the function with: { "userId": "1" } The test is written in the Lambda console as a JSON event (not a separate unit test file). If the test is successful, I'd see A green "Execution result: succeeded" banner.

The test displayed a "success" because the Lambda ran without crashing (the handler executed and returned). But the function's response was actually an access error/empty result, because the Lambda execution role doesn't have explicit DynamoDB read permissions

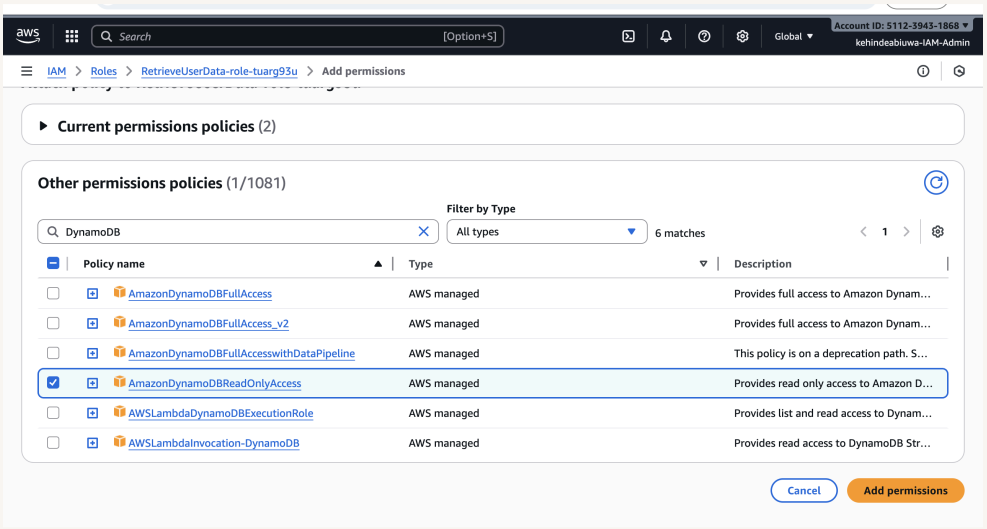


Function Permissions

To resolve the AccessDenied error, I attach read permissions to the Lambda execution role because DynamoDB blocks requests unless an identity-based policy explicitly allows actions like dynamodb:GetItem.

There were six DynamoDB permission policies I could choose from, but I didn't pick AWSLambdaInvocation-DynamoDB or AWSLambdaDynamoDBExecutionRole because they're for DynamoDB Streams workflows, not for reading items from a table: AWSLambdaDynamoDBExecutionRole → lets a Lambda read from a DynamoDB Stream (permissions like DescribeStream, GetRecords), which doesn't grant GetItem on my UserData table. AWSLambdaInvocation-DynamoDB → lets DynamoDB invoke my Lambda when stream events occur (a trigger permission), not my Lambda calling DynamoDB. For this project I just need my function to GetItem from the table, so I use AmazonDynamoDBReadOnlyAccess

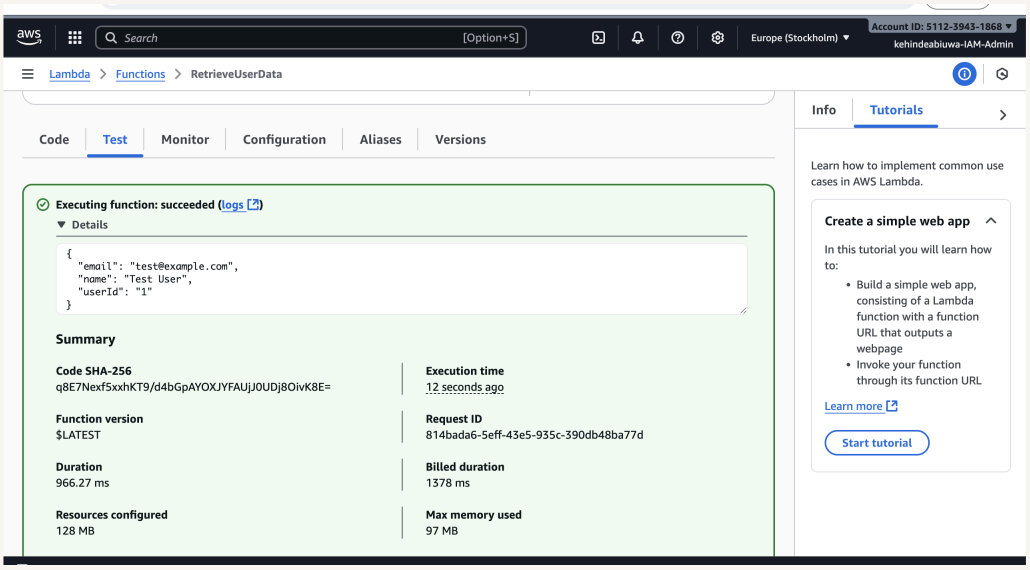
I also didn't pick AmazonDynamoDBFullAccess because it gives write/admin powers (create, update, delete tables/items, streams, backups) across DynamoDB—far beyond what my function needs and not aligned with least privilege. AmazonDynamoDBReadOnlyAccess was the right choice because my function only needs to read (GetItem, Query, Scan, DescribeTable). This policy unblocks the denied GetItem call without granting any destructive actions. (For production, I'd tighten further with a custom inline policy limited to the single UserData table.)


aws
Search
[Option+S]
Global
Account ID: 5112-3943-1868
kehindeabiuwa-IAM-Admin
IAM > Roles > RetrieveUserData-role-tuarg93u > Add permissions
Current permissions policies (2)
Other permissions policies (1/1081)
Filter by Type
DynamoDB
All types
6 matches
1
Policy name
Type
Description
AmazonDynamoDBFullAccess
AWS managed
Provides full access to Amazon Dynam...
AmazonDynamoDBFullAccess_v2
AWS managed
Provides full access to Amazon Dynam...
AmazonDynamoDBFullAccesswithDataPipeline
AWS managed
This policy is on a deprecation path. S...
AmazonDynamoDBReadOnlyAccess
AWS managed
Provides read only access to Amazon D...
AWSLambdaDynamoDBExecutionRole
AWS managed
Provides list and read access to Dynam...
AWSLambdaInvocation-DynamoDB
AWS managed
Provides read access to DynamoDB Str...
Cancel
Add permissions

Final Testing and Reflection

To validate my new permission settings, I re-ran the Lambda console test with the event { "userId": "1" }. The results were a green “Execution result: succeeded” and the payload: {"email":"test@example.com","name":"Test User","userId":"1"} because the function’s execution role now has read access (e.g., dynamodb:GetItem) to the UserData table, so the GetCommand call was authorized and returned the sample item.

Web apps are a popular use case of using Lambda and DynamoDB. For example, I could expose a “Get User Profile” API behind API Gateway that reads a user by userId from UserData. Other simple ways to apply this: Mobile app backend – fetch a user’s settings, saved items, or orders with a GET /users/{id} Lambda. E-commerce cart/checkout – quick reads of cart contents; use TTL to auto-expire abandoned carts. Game/leaderboard – get player stats or top scores with fast key lookups. Internal tools – Slack/Teams command that returns a user record from DynamoDB via Lambda. IoT device lookups – fetch the latest device state/config on demand. Scheduled jobs – EventBridge triggers Lambda nightly to read/report on specific items. Multi-tenant SaaS – read tenant-scoped data (e.g., tenantId#userId keys) with strict IAM.

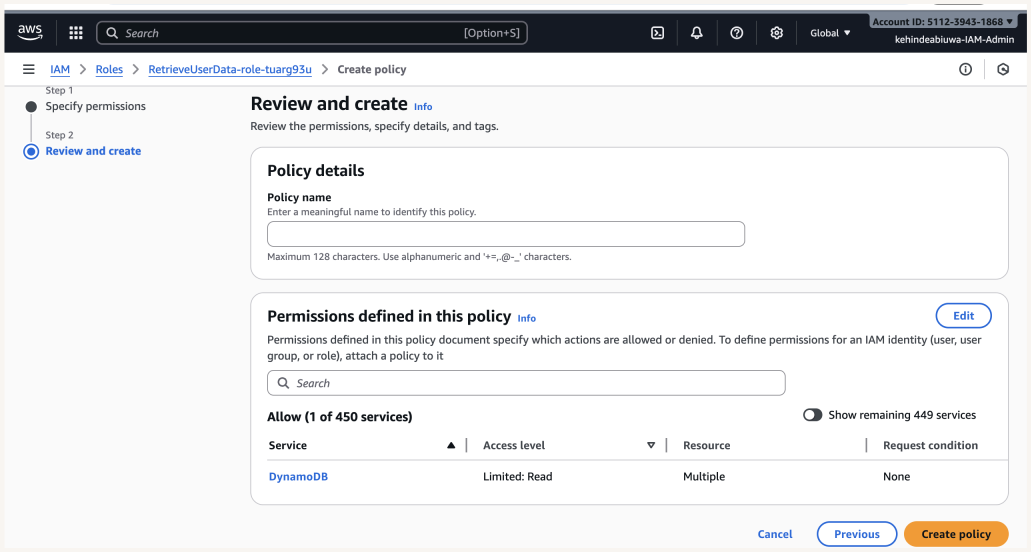

aws
Search
[Option+S]
Europe (Stockholm)
Account ID: 5112-3943-1868
kehindeabiuwa-IAM-Admin
Lambda > Functions > RetrieveUserData
Code
Test
Monitor
Configuration
Aliases
Versions
Executing function: succeeded (logs)
Details
{
"email": "test@example.com",
"name": "Test User",
"userId": "1"
}
Summary
Code SHA-256
q8E7Nexf5xxhKT9/d4bGpAYOXJYFAUjJ0UDj8OivK8E=
Execution time
12 seconds ago
Function version
$LATEST
Request ID
814bada6-5eff-43e5-935c-390db48ba77d
Duration
966.27 ms
Billed duration
1378 ms
Resources configured
128 MB
Max memory used
97 MB
Info
Tutorials
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial

Enahancing Security

For my project extension, I challenged myself to replace the broad AmazonDynamoDBReadOnlyAccess managed policy with a tight inline policy that only allows exactly what my function needs (e.g., dynamodb:GetItem on the single UserData table). This will enforce least privilege, remove unnecessary abilities like Scan/Query on all tables, reduce the blast radius if the function is misused, and make access easier to audit and reason about—while still letting the Lambda read the one item it needs.

To create the permission policy, I used the IAM console's Visual editor to add an inline policy on the Lambda role, because it's the quickest, least-error-prone way to pick only the read actions I need and scope them to the UserData table ARN (no broad access, no writes). I selected just the minimal read permissions (e.g., dynamodb:GetItem, plus metadata reads like DescribeTable if needed), reviewed the statement, and attached it directly to the role. This enforces least privilege while keeping the setup simple for this project.

When updating a Lambda funciton's permission policies, you could risk removing needed permissions and breaking data access. I validated that my Lambda function still works by re-running the saved test event in the Lambda console with: { "userId": "1" } and confirming: The banner shows "Execution result: succeeded." The payload returns my item CloudWatch logs contain a successful GetItem entry and no AccessDenied errors. This proves the tighter inline policy still allows the exact read my function needs.


aws
Search
[Option+S]
Global
Account ID: 5112-3943-1868
kehindeabiuwa-IAM-Admin
IAM > Roles > RetrieveUserData-role-tuarg93u > Create policy
Step 1
Specify permissions
Step 2
Review and create
Review and create Info
Review the permissions, specify details, and tags.
Policy details
Policy name
Enter a meaningful name to identify this policy.
Maximum 128 characters. Use alphanumeric and '+=,.@-_' characters.
Permissions defined in this policy Info
Edit
Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it
Search
Allow (1 of 450 services)
Show remaining 449 services
Service
Access level
Resource
Request condition
DynamoDB
Limited: Read
Multiple
None
Cancel
Previous
Create policy

The place to learn & showcase your skills

Check out nextwork.org for more projects